CO5_AT2 Pseudocode Writing Task (Algorithm Design)

Name: C. Himaja

Reg. No: 192373035

Source code

```
#include <stdio.h>
#include <limits.h>

#define MAX 10

int n;
int cost[MAX][MAX];

int visited[MAX];
int path[MAX];
int bestPath[MAX];

int minCost = INT_MAX;

/*
   Find the minimum edge cost in the cost matrix.
   It is used for calculating the lower bound.
*/
int findMinEdge()
{
   int min = INT_MAX;

   for (int i = 0; i < n; i++)
   {
```

```
        for (int j = 0; j < n; j++)
        {
            if (i != j && cost[i][j] < min)
            {
                min = cost[i][j];
            }
        }
    }

    return min;
}

/*
    TSP using Backtracking and Cost Bounding
*/
void tsp(int currentCity, int count,
         int currentCost, int minEdge)
{
    /*
        Base case:
        All cities have been visited.
    */
    if (count == n)
    {
        int totalCost =
            currentCost + cost[currentCity][0];

        /*
            Check whether this is the best tour.
        */
```

```
    if (totalCost < minCost)
    {
        minCost = totalCost;

        for (int i = 0; i < n; i++)
        {
            bestPath[i] = path[i];
        }

        bestPath[n] = 0;
    }

    return;
}

/*
    Try every unvisited city.
*/
for (int nextCity = 0; nextCity < n; nextCity++)
{
    if (!visited[nextCity])
    {
        int newCost =
            currentCost +
            cost[currentCity][nextCity];

        /*
            Calculate lower bound.
        */
        int remainingCities = n - count;
```

```
int lowerBound =
    newCost +
    remainingCities * minEdge;

/*
    Pruning:
    If the estimated minimum cost is
    already greater than the best cost,
    don't explore this branch.
*/
if (lowerBound >= minCost)
{
    continue;
}

/*
    Select the city.
*/
visited[nextCity] = 1;
path[count] = nextCity;

/*
    Recursive call.
*/
tsp(nextCity,
    count + 1,
    newCost,
    minEdge);
```

```
            /*
               Backtracking:
               Unselect the city.
            */
            visited[nextCity] = 0;
        }
    }
}

int main()
{
    printf("========================================\n");
    printf(" TRAVELING SALESMAN PROBLEM\n");
    printf(" Backtracking with Cost Bounding\n");
    printf("========================================\n");

    /*
       Input number of cities.
    */
    printf("\nEnter number of cities (2-%d): ", MAX);
    scanf("%d", &n);

    if (n < 2 || n > MAX)
    {
       printf("Invalid number of cities.\n");
       return 1;
    }

    /*
       Input cost matrix.
```

```
*/
printf("\nEnter the cost matrix:\n");

for (int i = 0; i < n; i++)
{
   for (int j = 0; j < n; j++)
   {
      scanf("%d", &cost[i][j]);

      if (cost[i][j] < 0)
      {
         printf("Cost cannot be negative.\n");
         return 1;
      }
   }
}

/*
   Initialize visited array.
*/
for (int i = 0; i < n; i++)
{
   visited[i] = 0;
}

/*
   Start from City 0.
*/
visited[0] = 1;
path[0] = 0;
```

```
/*
   Find minimum edge.
*/
int minEdge = findMinEdge();

/*
   Execute TSP.
*/
tsp(0, 1, 0, minEdge);

/*
   Display result.
*/
printf("\n----------------------------------------\n");

printf("Minimum Cost Tour:\n");

for (int i = 0; i <= n; i++)
{
   printf("City %d", bestPath[i]);

   if (i < n)
   {
      printf(" -> ");
   }
}

printf("\n");
```

```
    printf("\nMinimum Tour Cost = %d\n",
           minCost);

    printf("----------------------------------------\n");

    printf("\nBacktracking explores feasible tours\n");
    printf("and cost bounding prunes expensive paths.\n");

    printf("========================================\n");

    return 0;
}
```